

PART 4 –DQL(SELECT) QUERIES:

-- Top 5 Cities by Enrollments

```
230 SELECT TOP 5
231     s.City,
232     COUNT(e.EnrollmentID) AS TotalEnrollments
233 FROM Students s
234 INNER JOIN Enrollments e
235     ON s.StudentID = e.StudentID
236 GROUP BY s.City
237 ORDER BY TotalEnrollments DESC;
```

99 % 15 0 ↑ ↓ Ln: 237, Ch: 32 (199 chars, 8 lines) SPC CRLF UTF-8

Results Messages

	City	TotalEnrollments
1	Hyderabad	141
2	Ahmedabad	135
3	Goa	131
4	Jaipur	117
5	Delhi	116

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (70) LearnProDB 00:00:00 Row: 1, Col: 1 5 rows

--Students Enrolled in More Than 2 Courses

```
237 --Students Enrolled in More Than 2 Courses
238
239 SELECT
240     s.StudentID,
241     s.FullName,
242     COUNT(e.CourseID) AS TotalCourses
243 FROM Students s
244 INNER JOIN Enrollments e
245     ON s.StudentID = e.StudentID
246 GROUP BY
247     s.StudentID,
248     s.FullName
249 HAVING COUNT(e.CourseID) > 2;
```

99 % 16 0 ↑ ↓ Ln: 249, Ch: 30 (232 chars, 11 lines) SPC CRLF UTF-8

Results Messages

	StudentID	FullName	TotalCourses
1	1010	Student_10	4
2	1012	Student_12	7
3	1014	Student_14	3
4	1019	Student_19	3
5	1020	Student_20	4
6	1021	Student_21	3
7	1039	Student_39	3
8	1043	Student_43	3
9	1060	Student_60	3
10	1084	Student_84	4
11	1087	Student_87	3
12	1101	Student_...	4
13	1102	Student_...	3

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (95) LearnProDB 00:00:00 Row: 1, Col: 1 143 rows

--Students with No Payment

```

252 | SELECT
253 |     s.StudentID,
254 |     s.FullName,
255 |     e.PaymentStatus
256 | FROM Students s
257 | INNER JOIN Enrollments e
258 |     ON s.StudentID = e.StudentID
259 | LEFT JOIN Payments p
260 |     ON e.EnrollmentID = p.EnrollmentID
261 | WHERE e.PaymentStatus = 'Unpaid';

```

99 % 15 0 ↑ ↓ Ln: 261, Ch: 34 (236 chars, 10 lines) SPC CRLF UTF-8

Results Messages

	StudentID	FullName	PaymentStatus
1	1008	Student_8	Unpaid
2	1010	Student_10	Unpaid
3	1012	Student_12	Unpaid
4	1012	Student_12	Unpaid
5	1019	Student_19	Unpaid
6	1024	Student_24	Unpaid
7	1025	Student_25	Unpaid
8	1028	Student_28	Unpaid
9	1029	Student_29	Unpaid
10	1035	Student_35	Unpaid
11	1036	Student_36	Unpaid
12	1038	Student_38	Unpaid
13	1039	Student_39	Unpaid
14	1042	Student_42	Unpaid
15	1047	Student_47	Unpaid

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 373 rows

-- Highest revenue generating month.

```

252 | SELECT
253 |     s.StudentID,
254 |     s.FullName,
255 |     e.PaymentStatus
256 | FROM Students s
257 | INNER JOIN Enrollments e
258 |     ON s.StudentID = e.StudentID
259 | LEFT JOIN Payments p
260 |     ON e.EnrollmentID = p.EnrollmentID
261 | WHERE e.PaymentStatus = 'Unpaid';
262 |
263 | --Highest Revenue Month
264 | SELECT TOP (1)
265 |     DATENAME(MONTH, PaymentDate) AS MonthName,
266 |     SUM(AmountPaid) AS TotalRevenue
267 | FROM Payments
268 | GROUP BY DATENAME(MONTH, PaymentDate),
269 |     MONTH(PaymentDate)
270 | ORDER BY TotalRevenue DESC;
271 |

```

99 % 15 0 ↑ ↓ Ln: 271, Ch: 1 (214 chars, 7 lines) SPC CRLF UTF-8

Results Messages

	MonthName	TotalRevenue
1	March	4465133.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 1 rows

-- Top 3 most popular courses.

```
260 ON e.EnrollmentID = p.EnrollmentID
261 WHERE e.PaymentStatus = 'Unpaid';
262
263 -- Highest revenue generating month.
264 SELECT TOP (1)
265     DATENAME(MONTH, PaymentDate) AS MonthName,
266     SUM(AmountPaid) AS TotalRevenue
267 FROM Payments
268 GROUP BY DATENAME(MONTH, PaymentDate),
269     MONTH(PaymentDate)
270 ORDER BY TotalRevenue DESC;
271 -- Top 3 most popular courses.
272 SELECT TOP (3)
273     c.CourseID,
274     c.CourseName,
275     COUNT(e.EnrollmentID) AS TotalEnrollments
276 FROM Courses c
277 INNER JOIN Enrollments e
278     ON c.CourseID = e.CourseID
279 GROUP BY
280     c.CourseID,
281     c.CourseName
282 ORDER BY TotalEnrollments DESC;
```

Ln: 282, Ch: 32 (249 chars, 11 lines) SPC CRLF UTF-8

	CourseID	CourseName	TotalEnrollments
1	508	Generative AI	129
2	509	Cloud Foundations	129
3	505	Python Full Stack	128

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 3 rows

--. Enrollments with discount above average discount.

```
287 --. Enrollments with discount above average discount.
288
289 SELECT
290     e.EnrollmentID,
291     s.FullName,
292     c.CourseName,
293     e.Discount
294 FROM Enrollments e
295 INNER JOIN Students s
296     ON e.StudentID = s.StudentID
297 INNER JOIN Courses c
298     ON e.CourseID = c.CourseID
299 WHERE e.Discount >
300 (
301     SELECT AVG(Discount)
302     FROM Enrollments
303 )
304 ORDER BY e.Discount DESC;
```

Ln: 292, Ch: 18 SPC CRLF UTF-8

	EnrollmentID	FullName	CourseName	Discount
1	2183	Student_1	SQL Mastery	30.00
2	3018	Student_10	Data Analytics	30.00
3	2849	Student_17	SQL Mastery	30.00
4	2988	Student_19	Power BI	30.00
5	2799	Student_21	Python Full Stack	30.00
6	2619	Student_42	SQL Mastery	30.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 532 rows

--States with more than 10 paid enrollments.

```

307
308 SELECT
309     s.State,
310     COUNT(e.EnrollmentID) AS TotalPaidEnrollments
311 FROM Students s
312 INNER JOIN Enrollments e
313     ON s.StudentID = e.StudentID
314 WHERE e.PaymentStatus = 'Paid'
315 GROUP BY s.State
316 HAVING COUNT(e.EnrollmentID) > 10
317 ORDER BY TotalPaidEnrollments ;

```

Ln: 317, Ch: 32 (269 chars, 11 lines) SPC CRLF UTF-8

Results Messages

	State	TotalPaidEnrollments
1	Rajasthan	30
2	Tamil Nadu	32
3	Delhi	33
4	Karnataka	33
5	West Bengal	41
6	Goa	47
7	Gujarat	52
8	Telangana	52
9	Maharashtra	80

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 9 rows

PART 5 -JOINS(ALL TYPES)

--Students + Enrollments + Courses (STUDENT NAME + COURSE NAME + TRAINER NAME)

```

320
321 --Students + Enrollments + Courses (STUDENT NAME + COURSE NAME + TRAINER NAME)
322
323 SELECT
324     s.FullName,
325     c.CourseName,
326     t.TrainerName
327 FROM Students s
328 INNER JOIN Enrollments e
329     ON s.StudentID = e.StudentID
330 INNER JOIN Courses c
331     ON e.CourseID = c.CourseID
332 INNER JOIN TrainerCourseMapping tm
333     ON c.CourseID = tm.CourseID
334 INNER JOIN Trainers t
335     ON tm.TrainerID = t.TrainerID
336 ORDER BY
337     s.FullName,
338     c.CourseName;

```

Ln: 338, Ch: 18 (365 chars, 16 lines) SPC CRLF UTF-8

Results Messages

	FullName	CourseName	TrainerName
1	Student_1	Generative AI	Trainer_9
2	Student_1	Generative AI	Trainer_2
3	Student_1	Generative AI	Trainer_1
4	Student_1	SQL Mastery	Trainer_5
5	Student_1	SQL Mastery	Trainer_6
6	Student_1	SQL Mastery	Trainer_9
7	Student_1	SQL Mastery	Trainer_6
8	Student_1	Business A	Trainer_2

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 3,525 rows

--Courses + TrainerCourseMapping + Trainers(TRAINER PERFORMANCE WITH NUMBER OF ASSIGNED COURSE)

```
341
342 SELECT
343     t.TrainerID,
344     t.TrainerName,
345     t.Exp column TrainerName(varchar, not null) courses
346     COUNT
347 FROM Trainers t
348 INNER JOIN TrainerCourseMapping tm
349     ON t.TrainerID = tm.TrainerID
350 INNER JOIN Courses c
351     ON tm.CourseID = c.CourseID
352 GROUP BY
353     t.TrainerID,
354     t.TrainerName,
355     t.Expertise
```

99 % 15 0 361, Ch: 15 SPC CRLF UTF-8

	TrainerID	TrainerName	Expertise	TotalAssignedCourses
1	4001	Trainer_1	Power BI	6
2	4008	Trainer_8	Python	6
3	4006	Trainer_6	Power BI	5
4	4002	Trainer_2	Python	5
5	4009	Trainer_9	Power BI	4
6	4010	Trainer_10	Cloud	1
7	4003	Trainer_3	AI	1
8	4004	Trainer_4	Power BI	1

	CourseName	Revenue
1	Data Analytics	4747754.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (57) LearnProDB 00:00:00 Row: 1, Col: 1 19 rows

--Enrollments + Payments(REVENUE PER COURSE)

```
362 --Enrollments + Payments(REVENUE PER COURSE)
363 SELECT
364     c.CourseName,
365     SUM(p.AmountPaid) AS Revenue
366 FROM Courses c
367 INNER JOIN Enrollments e
368     ON c.CourseID=e.CourseID
369 INNER JOIN Payments p
370     ON e.EnrollmentID=p.EnrollmentID
371 GROUP BY
372     c.CourseName
373 ORDER BY
374     Revenue DESC;
375 -- Revenue per course
376
377
```

99 % 45 0 375, Ch: 1 (243 chars, 12 lines) SPC CRLF UTF-8

	CourseName	Revenue
1	Data Analytics	4747754.00
2	Power BI	4469093.00
3	Data Science	4458602.00
4	Generative AI	4334813.00
5	SQL Mastery	4261140.00
6	Cloud Foundations	4244354.00
7	Python Full Stack	4143598.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (84) LearnProDB 00:00:00 Row: 1, Col: 1 10 rows

-- Revenue per course

```

376 -- Revenue per course
377
378 SELECT
379     c.CourseName,
380     SUM(p.AmountPaid) AS TotalRevenue
381 FROM Courses c
382 INNER JOIN Enrollments e
383     ON c.CourseID = e.CourseID
384 INNER JOIN Payments p
385     ON e.EnrollmentID = p.EnrollmentID
386 GROUP BY c.CourseName
387 ORDER BY TotalRevenue DESC;
388
389 -- City wise + course wise revenue summary

```

99 % 45 0 Ln: 388, Ch: 1 (255 chars, 10 lines) SPC CRLF UTF-8

Results Messages

	CourseName	TotalRevenue
1	Data Analytics	4747754.00
2	Power BI	4469093.00
3	Data Science	4458602.00
4	Generative AI	4334813.00
5	SQL Mastery	4261140.00
6	Cloud Foundations	4244354.00
7	Python Full Stack	4143598.00
8	Machine Learning	3880273.00
9	Business Analytics	3741912.00
10	Excel Power User	3476370.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (84) LearnProDB 00:00:00 Row: 1, Col: 1 10 rows

-- City wise + course wise revenue summary

```

388
389 -- City wise + course wise revenue summary
390
391 SELECT
392     s.City,
393     c.CourseName,
394     COUNT(e.EnrollmentID) AS TotalEnrollments,
395     SUM(p.AmountPaid) AS TotalRevenue,
396     AVG(p.AmountPaid) AS AverageRevenue
397 FROM Students s
398 INNER JOIN Enrollments e
399     ON s.StudentID = e.StudentID
400 INNER JOIN Courses c
401     ON e.CourseID = c.CourseID
402 INNER JOIN Payments p

```

99 % 45 0 Ln: 410, Ch: 1 (455 chars, 19 lines) OVR SPC CRLF UTF-8

Results Messages

	City	CourseName	TotalEnrollments	TotalRevenue	AverageRevenue
1	Ahmedabad	Python Full Stack	21	702447.00	33449.857142
2	Ahmedabad	Power BI	16	642560.00	40160.000000
3	Ahmedabad	Excel Power User	14	567968.00	40569.142857
4	Ahmedabad	Data Science	11	485895.00	44172.272727
5	Ahmedabad	Machine Learning	12	482512.00	40209.333333
6	Ahmedabad	Business Analytics	14	473401.00	33814.357142
7	Ahmedabad	Data Analytics	13	462347.00	35565.153846
8	Ahmedabad	Generative AI	12	433527.00	36127.250000
9	Ahmedabad	SQL Mastery	10	395417.00	39541.700000
10	Ahmedabad	Cloud Foundations	12	391358.00	32613.166666
11	Bangalore	Data Analytics	9	455578.00	50619.777777

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (84) LearnProDB 00:00:00 Row: 1, Col: 1 100 rows

PART 6 – GROUP BY + HAVING + AGGREGATION

--Monthly Revenue

My_SQL_Assi...T\DELL (84)*

```

413 --Monthly Revenue
414
415 SELECT
416     YEAR(PaymentDate) AS Year,
417     MONTH(PaymentDate) AS Month,
418     COUNT(PaymentID) AS TotalPayments,
419     SUM(AmountPaid) AS TotalRevenue,
420     AVG(AmountPaid) AS AveragePayment,
421     MIN(AmountPaid) AS MinimumPayment,
422     MAX(AmountPaid) AS MaximumPayment
423 FROM Payments
424 GROUP BY
425     YEAR(PaymentDate),
426     MONTH(PaymentDate)
427 ORDER BY

```

99 % 45 0

Ln: 430, Ch: 1 (377 chars, 15 lines) OVR SPC CRLF UTF-8

Results Messages

	Year	Month	TotalPayments	TotalRevenue	AveragePayment	MinimumPayment	MaximumPayment
1	2025	1	31	1080815.00	34865.000000	1168.00	69711.00
2	2025	2	28	893113.00	31896.892857	1407.00	62785.00
3	2025	3	31	1211624.00	39084.645161	7837.00	65220.00
4	2025	4	30	1217986.00	40599.533333	7194.00	68475.00
5	2025	5	31	999518.00	32242.516129	2042.00	67600.00
6	2025	6	30	974654.00	32488.466666	5495.00	69123.00
7	2025	7	31	1130095.00	36454.677419	2581.00	68931.00
8	2025	8	31	1236910.00	39900.322580	3407.00	69526.00
9	2025	9	30	1004515.00	33483.833333	3385.00	68369.00
10	2025	10	31	1205190.00	38877.096774	1893.00	68653.00
11	2025	11	30	1003845.00	33461.500000	229.00	69263.00

-- Enrollment counts per course

My_SQL_Assi...T\DELL (84)*

```

432
433 SELECT
434     c.CourseID,
435     c.CourseName,
436     COUNT(e.EnrollmentID) AS TotalEnrollments
437 FROM Courses c
438 INNER JOIN Enrollments e
439     ON c.CourseID = e.CourseID
440 GROUP BY
441     c.CourseID,
442     c.CourseName
443 ORDER BY
444     TotalEnrollments DESC;
445
446 -- Average fee per category

```

99 % 45 0

Ln: 444, Ch: 27 (248 chars, 12 lines) OVR SPC CRLF UTF-8

Results Messages

	CourseID	CourseName	TotalEnrollments
1	508	Generative AI	129
2	509	Cloud Foundations	129
3	505	Python Full Stack	128
4	503	Data Science	127
5	506	SQL Mastery	123
6	501	Data Analytics	123
7	504	Power BI	121
8	502	Business Analytics	111
9	507	Machine Learning	107
10	510	Excel Power User	103

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (84) LearnProDB 00:00:00 Row: 1, Col: 1 10 rows

-- Average fee per category

My_SQL_Assi...T\DELL (84)*

```

445
446 -- Average fee per category
447
448 SELECT
449     Category,
450     COUNT(CourseID) AS TotalCourses,
451     AVG(CourseFee) AS AverageFee,
452     MIN(CourseFee) AS MinimumFee,
453     MAX(CourseFee) AS MaximumFee
454 FROM Courses
455 GROUP BY Category
456 ORDER BY AverageFee DESC;
457
458 --Trainer Satisfaction Averages
459

```

99 % 45 0

Ln: 457, Ch: 1 (225 chars, 9 lines) OVR SPC CRLF UTF-8

Results Messages

	Category	TotalCourses	AverageFee	MinimumFee	MaximumFee
1	AI	3	70000.000000	65000.00	75000.00
2	Programming	1	55000.000000	55000.00	55000.00
3	Analytics	2	47500.000000	45000.00	50000.00
4	Cloud	1	40000.000000	40000.00	40000.00
5	BI	1	30000.000000	30000.00	30000.00
6	Databases	1	25000.000000	25000.00	25000.00
7	Productivity	1	20000.000000	20000.00	20000.00

--Trainer Satisfaction Averages

My_SQL_Assi...T\DELL (84)*

```

457
458 --Trainer Satisfaction Averages
459
460 SELECT
461     Expertise,
462     COUNT(TrainerID) AS TotalTrainers,
463     AVG(SatisfactionScore) AS AverageSatisfaction,
464     MIN(SatisfactionScore) AS MinimumRating,
465     MAX(SatisfactionScore) AS MaximumRating
466 FROM Trainers
467 GROUP BY Expertise
468 ORDER BY AverageSatisfaction DESC;
469
470 -- State wise registration patterns
471

```

99 % 45 0

Ln: 469, Ch: 1 (278 chars, 9 lines) OVR SPC CRLF UTF-8

Results Messages

	Expertise	Total Trainers	Average Satisfaction	Minimum Rating	Maximum Rating
1	Power BI	4	4.355000	4.19	4.48
2	AI	2	4.135000	3.87	4.40
3	Cloud	1	3.990000	3.99	3.99
4	SQL	1	3.840000	3.84	3.84
5	Python	2	3.620000	3.50	3.74

-- State wise registration patterns

My_SQL_Assi...T\DELL (84))*

```
469
470 -- State wise registration patterns
471
472 SELECT
473     s.State,
474     COUNT(s.StudentID) AS TotalRegistrations
475 FROM Students s
476 GROUP BY s.State
477 ORDER BY TotalRegistrations DESC;
478
479 --PART 7 - SUBQUERIES + CTEs
480
481 -- Courses with above average enrollments
482
483 SELECT
```

99 % 45 0 ↑ ↓ Ln: 479, Ch: 1 (140 chars, 7 lines) OVR SPC CRLF UTF-8

Results Messages

	State	TotalRegistrations
1	Maharashtra	166
2	Telangana	99
3	Gujarat	99
4	Karnataka	94
5	Tamil Nadu	94
6	Delhi	92
7	Goa	90
8	Rajasthan	84
9	West Bengal	82

PART 7 – SUBQUERIES+CTEs

-- Courses with above average enrollments

My_SQL_Assi...T\DELL (84))*

```
479 --PART 7 - SUBQUERIES + CTEs
480
481 -- Courses with above average enrollments
482
483 SELECT
484     c.CourseID,
485     c.CourseName,
486     COUNT(e.EnrollmentID) AS TotalEnrollments
487 FROM Courses c
488 INNER JOIN Enrollments e
489 ON c.CourseID = e.CourseID
490 GROUP BY
491     c.CourseID,
492     c.CourseName
493 HAVING COUNT(e.EnrollmentID) >
```

99 % 45 0 Ln: 504, C

Results Messages

	CourseID	CourseName	TotalEnrollments
1	501	Data Analytics	123
2	504	Power BI	121
3	505	Python Full Stack	128
4	508	Generative AI	129
5	503	Data Science	127
6	509	Cloud Foundations	129
7	506	SQL Mastery	123

--Students Enrolled in the Most Expensive Course

My_SQL_Assi...T\DELL (84)*

```
503
504 --Students Enrolled in the Most Expensive Course
505
506 SELECT
507     s.StudentID,
508     s.FullName,
509     c.CourseName,
510     c.CourseFee
511 FROM Students s
512 INNER JOIN Enrollments e
513 ON s.StudentID=e.StudentID
514 INNER JOIN Courses c
515 ON e.CourseID=c.CourseID
516 WHERE c.CourseFee=
517 (
```

99 % 45 0

Results Messages

	StudentID	FullName	CourseName	CourseFee
1	1001	Student_1	Generative AI	75000.00
2	1014	Student_14	Generative AI	75000.00
3	1019	Student_19	Generative AI	75000.00
4	1050	Student_50	Generative AI	75000.00
5	1066	Student_66	Generative AI	75000.00
6	1069	Student_69	Generative AI	75000.00
7	1070	Student_70	Generative AI	75000.00
8	1087	Student_87	Generative AI	75000.00
9	1102	Student_102	Generative AI	75000.00
10	1102	Student_102	Generative AI	75000.00
11	1111	Student_111	Generative AI	75000.00

--Highest Discount Enrollment Details

My_SQL_Assi...T\DELL (84))*

```
521
522 --Highest Discount Enrollment Details
523
524 SELECT
525     e.EnrollmentID,
526     s.FullName,
527     c.CourseName,
528     e.Discount
529 FROM Enrollments e
530 INNER JOIN Students s
531 ON e.StudentID=s.StudentID
532 INNER JOIN Courses c
533 ON e.CourseID=c.CourseID
534 WHERE e.Discount=
535 (
536     SELECT MAX(Discount)
537     FROM Enrollments
538 );
539
540 --Revenue Trend using CTE (Last 6 Months)
```

99 % 45 0

Results Messages

	EnrollmentID	FullName	CourseName	Discount
1	1	Student_1	Data Analytics	50.00

--Revenue Trend using CTE (Last 6 Months)

My_SQL_Assi...T\DELL (84))*

```

539
540 --Revenue Trend using CTE (Last 6 Months)
541
542 WITH MonthlyRevenue AS
543 (
544     SELECT
545         YEAR(PaymentDate) AS Year,
546         MONTH(PaymentDate) AS Month,
547         SUM(AmountPaid) AS TotalRevenue
548     FROM Payments
549     WHERE PaymentDate >= DATEADD(MONTH,-6,GETDATE())
550     GROUP BY
551         YEAR(PaymentDate),
552         MONTH(PaymentDate)
553 )
554
555 SELECT *
556 FROM MonthlyRevenue
557 ORDER BY
558     Year,

```

99 % 45 0

Ln: 561

Results Messages

	Year	Month	TotalRevenue
1	2026	1	294805.00
2	2026	2	1030322.00
3	2026	3	1080211.00
4	2026	4	1196849.00
5	2026	5	1119375.00
6	2026	6	1245082.00
7	2026	7	1007205.00

Query executed successfully

DESKTOP-V9S50ET\SQLEXPRESS | DESKTOP-V9S50ET\DELL (84)

PART 8 – WINDOW FUNCTION

--RANK() – Revenue Per Month

My_SQL_Assi...T\DELL (84))*

```
562
563 --RANK() - Revenue Per Month
564
565 SELECT
566     YEAR(PaymentDate) AS Year,
567     MONTH(PaymentDate) AS Month,
568     SUM(AmountPaid) AS TotalRevenue,
569     RANK() OVER(ORDER BY SUM(AmountPaid) DESC) AS RevenueRank
570 FROM Payments
571 GROUP BY
572     YEAR(PaymentDate),
573     MONTH(PaymentDate)
574 ORDER BY RevenueRank;
575
```

99 % 45 0

Results Messages

	Year	Month	TotalRevenue	RevenueRank
1	2026	6	1245082.00	1
2	2025	8	1236910.00	2
3	2026	12	1225311.00	3
4	2026	1	1222495.00	4
5	2025	4	1217986.00	5
6	2025	3	1211624.00	6
7	2025	10	1205190.00	7
8	2026	4	1196849.00	8
9	2025	12	1183514.00	9
10	2026	11	1151618.00	10
11	2025	7	1130095.00	11
12	2026	8	1126529.00	12

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL

-- DENSE_RANK() top 3 courses per month

My_SQL_Assi...T\DELL (84)*

```
575
576 -- DENSE_RANK() top 3 courses per month
577
578 WITH CourseEnrollment AS
579 (
580     SELECT
581         YEAR(e.EnrollmentDate) AS Year,
582         MONTH(e.EnrollmentDate) AS Month,
583         c.CourseName,
584         COUNT(e.EnrollmentID) AS TotalEnrollments
585     FROM Enrollments e
586     INNER JOIN Courses c
587     ON e.CourseID = c.CourseID
588     GROUP BY
```

99 % 45 0

Results Messages

	Year	Month	CourseName	TotalEnrollments	CourseRank
1	2025	1	Power BI	12	1
2	2025	1	SQL Mastery	11	2
3	2025	1	Generative AI	10	3
4	2025	1	Data Science	10	3
5	2025	1	Business Analytics	10	3
6	2025	2	Power BI	14	1
7	2025	2	Data Science	12	2
8	2025	2	Excel Power User	11	3
9	2025	3	Python Full Stack	15	1
10	2025	3	Data Analytics	14	2
11	2025	3	Generative AI	13	3
12	2025	4	Machine Learning	14	1

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (84) LearnProDB

--LAG() – Month-over-Month Revenue Change

My_SQL_Assi...T\DELL (84))*

```

605 WHERE CourseRank <= 3;
606
607 --LAG() - Month-over-Month Revenue Change
608
609 WITH MonthlyRevenue AS
610 (
611     SELECT
612         YEAR(PaymentDate) AS Year,
613         MONTH(PaymentDate) AS Month,
614         SUM(AmountPaid) AS TotalRevenue
615     FROM Payments
616     GROUP BY
617         YEAR(PaymentDate),
618         MONTH(PaymentDate)

```

99 % 45 0

Ln: 629, C

Results Messages

	Year	Month	TotalRevenue	PreviousMonthRevenue	RevenueDifference
1	2025	1	1080815.00	NULL	NULL
2	2025	2	893113.00	1080815.00	-187702.00
3	2025	3	1211624.00	893113.00	318511.00
4	2025	4	1217986.00	1211624.00	6362.00
5	2025	5	999518.00	1217986.00	-218468.00
6	2025	6	974654.00	999518.00	-24864.00
7	2025	7	1130095.00	974654.00	155441.00
8	2025	8	1236910.00	1130095.00	106815.00
9	2025	9	1004515.00	1236910.00	-232395.00
10	2025	10	1205190.00	1004515.00	200675.00
11	2025	11	1003845.00	1205190.00	-201345.00
12	2025	12	1183514.00	1003845.00	179669.00

Query executed successfully.

DESKTOP-V8S5AET\SQLEXPRESS | DESKTOP-V8S5AET\DELL (84) |

--LEAD() – Forecast Next Month Trend

My_SQL_Assi...T\DELL (84))*

```
629
630 --LEAD() - Forecast Next Month Trend
631
632 WITH MonthlyRevenue AS
633 (
634     SELECT
635         YEAR(PaymentDate) AS Year,
636         MONTH(PaymentDate) AS Month,
637         SUM(AmountPaid) AS TotalRevenue
638     FROM Payments
639     GROUP BY
640         YEAR(PaymentDate),
641         MONTH(PaymentDate)
642 )
```

99 % 45 0

Results Messages

	Year	Month	TotalRevenue	NextMonthRevenue
1	2025	1	1080815.00	893113.00
2	2025	2	893113.00	1211624.00
3	2025	3	1211624.00	1217986.00
4	2025	4	1217986.00	999518.00
5	2025	5	999518.00	974654.00
6	2025	6	974654.00	1130095.00
7	2025	7	1130095.00	1236910.00
8	2025	8	1236910.00	1004515.00
9	2025	9	1004515.00	1205190.00
10	2025	10	1205190.00	1003845.00
11	2025	11	1003845.00	1183514.00
12	2025	12	1183514.00	1222495.00

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL

-- ROW_NUMBER() for duplicate detection -- again check

My_SQL_Assi...T\DELL (84))*

```
649 FROM MonthlyRevenue;  
650  
651 -- ROW_NUMBER() for duplicate detection  
652  
653  
654 SELECT  
655     StudentID,  
656     FullName,  
657     Email,  
658     ROW_NUMBER() OVER  
659     (  
660         PARTITION BY Email  
661         ORDER BY StudentID  
662     ) AS RowNum
```

99 % 45 0

Results Messages

	StudentID	FullName	Email	RowNum
1	1001	Student_1	student1@example.com	1
2	1010	Student_10	student10@example.com	1
3	1100	Student_100	student100@example.com	1
4	1101	Student_101	student101@example.com	1
5	1102	Student_102	student102@example.com	1
6	1103	Student_103	student103@example.com	1
7	1104	Student_104	student104@example.com	1
8	1105	Student_105	student105@example.com	1
9	1106	Student_106	student106@example.com	1

PART 9 - VIEWS

vw_MonthlyRevenue

My_SQL_Assi...T\DELL (84))*

```
682
683  --PART 9 - VIEWS
684
685  --vw_MonthlyRevenue
686
687  CREATE VIEW vw_MonthlyRevenue
688  AS
689  SELECT
690      YEAR(p.PaymentDate) AS Year,
691      MONTH(p.PaymentDate) AS Month,
692      COUNT(p.PaymentID) AS TotalPayments,
693      SUM(p.AmountPaid) AS TotalRevenue,
694      AVG(p.AmountPaid) AS AverageRevenue,
695      MIN(p.AmountPaid) AS MinimumPayment,
```

99 % 45 0

Results

Messages

	Year	Month	TotalPayments	TotalRevenue	AverageRevenue	MinimumPayment	MaximumPayment
1	2027	8	31	948171.00	30586.161290	555.00	66922.00
2	2026	7	31	1007205.00	32490.483870	1468.00	66344.00
3	2026	1	31	1222495.00	39435.322580	2251.00	69351.00
4	2025	6	30	974654.00	32488.466666	5495.00	69123.00
5	2028	4	14	367722.00	26265.857142	8593.00	49377.00
6	2026	10	31	982141.00	31681.967741	3981.00	67306.00
7	2027	11	30	1055820.00	35194.000000	3293.00	67359.00
8	2026	4	30	1196849.00	39894.966666	1044.00	68178.00
9	2025	3	31	1211624.00	39084.645161	7837.00	65220.00
10	2028	1	31	1032553.00	33308.161290	368.00	69795.00
11	2025	9	30	1004515.00	33483.833333	3385.00	68369.00
12	2027	6	30	972695.00	32423.166666	1737.00	59635.00

--vw_TrainerPerformance

My_SQL_Assi...T\DELL (84))*

```

703 SELECT * FROM vw_MonthlyRevenue;
704
705 --vw_TrainerPerformance
706
707 CREATE VIEW vw_TrainerPerformance
708 AS
709 SELECT
710     t.TrainerID,
711     t.TrainerName,
712     t.Expertise,
713     COUNT(DISTINCT tm.CourseID) AS TotalCourses,
714     COUNT(e.StudentID) AS TotalStudents,
715     SUM(p.AmountPaid) AS TotalRevenue,
716     AVG(t.SatisfactionScore) AS AverageSatisfaction

```

99 % 45 0

Results Messages

	TrainerID	TrainerName	Expertise	TotalCourses	TotalStudents	TotalRevenue	AverageSatisfaction
1	4005	Trainer_5	AI	1	123	4261140.00	3.870000
2	4008	Trainer_8	Python	4	698	24700056.00	3.740000
3	4002	Trainer_2	Python	4	580	20446664.00	3.500000
4	4003	Trainer_3	AI	1	107	3880273.00	4.400000
5	4006	Trainer_6	Power BI	3	581	19719374.00	4.340000
6	4009	Trainer_9	Power BI	4	470	16218138.00	4.480000
7	4001	Trainer_1	Power BI	5	708	25088244.00	4.190000
8	4004	Trainer_4	Power BI	1	129	4244354.00	4.410000
9	4010	Trainer_10	Cloud	1	129	4244354.00	3.990000

PART 10 – INDEXES

-: Students.Email

My_SQL_Assi...T\DELL (84))*

```

730  SELECT * FROM vw_TrainerPerformance;
731
732  -- SECTION 10: INDEXES
733  -- Index 1 : Students.Email
734  CREATE INDEX IX_Students_Email
735  ON Students(Email);
736  --FOR CHECKING INDEX
737  EXEC sp_helpindex 'Students';
738
739  -- Index 2 : Courses.CourseName
740  CREATE INDEX IX_Courses_CourseName
741  ON Courses(CourseName);
742  --FOR CHECKING INDEX
743  EXEC sp_helpindex 'Courses';

```

99 % 45 0

Results Messages

	index_name	index_description	index_keys
1	PK_Students__32C52A7987764253	clustered, unique, primary key located on PRIMARY	StudentID
2	UQ_Students__5C7E359E25D3FC13	nonclustered, unique, unique key located on PRIMA...	Phone
3	UQ_Students__A9D10534E716A634	nonclustered, unique, unique key located on PRIMA...	Email

-- Index 2 : Courses.CourseName

My_SQL_Assi...T\DELL (84))*

```

731
732 -- SECTION 10: INDEXES
733 -- Index 1 : Students.Email
734 CREATE INDEX IX_Students_Email
735 ON Students(Email);
736 --FOR CHECKING INDEX
737 EXEC sp_helpindex 'Students';
738
739 -- Index 2 : Courses.CourseName
740 CREATE INDEX IX_Courses_CourseName
741 ON Courses(CourseName);
742 --FOR CHECKING INDEX
743 EXEC sp_helpindex 'Courses';
744 -- Index 3 : Enrollment Date

```

99 % 45 0

Results Messages

	index_name	index_description	index_keys
1	IX_Courses_CourseName	nonclustered located on PRIMARY	CourseName
2	PK_Courses__C92D7187D9756E1C	clustered, unique, primary key located on PRIMARY	CourseID
3	UQ_Courses__9526E27774234885	nonclustered, unique, unique key located on PRIMA...	CourseName

-- Index 3 : Enrollment Date

```

744 -- Index 3 : Enrollment Date
745 CREATE INDEX IX_Enrollments_EnrollmentDate
746 ON Enrollments(EnrollmentDate);
747 --FOR CHECKING INDEX
748 EXEC sp_helpindex 'Enrollments';
749 -- Index 4 : Composite Index(StudentID,CourseID)
750 CREATE INDEX IX_Enrollments_Student_Course

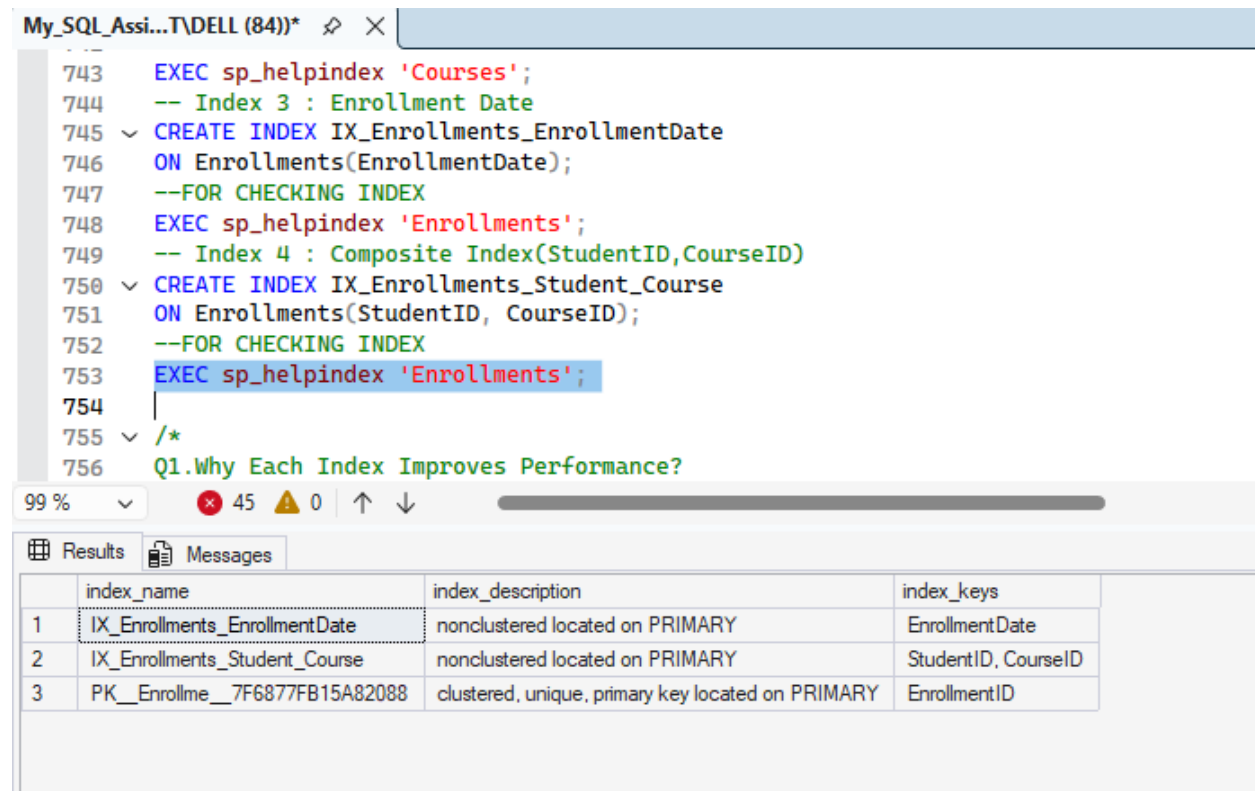
```

99 % 45 0

Results Messages

	index_name	index_description	index_keys
1	IX_Enrollments_EnrollmentDate	nonclustered located on PRIMARY	EnrollmentDate
2	IX_Enrollments_Student_Course	nonclustered located on PRIMARY	StudentID, CourseID
3	PK__Enrollme__7F6877FB15A82088	clustered, unique, primary key located on PRIMARY	EnrollmentID

-- Index 4 : Composite Index(StudentID,CourseID)



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a script with the following commands:

```
743 EXEC sp_helpindex 'Courses';
744 -- Index 3 : Enrollment Date
745 CREATE INDEX IX_Enrollments_EnrollmentDate
746 ON Enrollments(EnrollmentDate);
747 --FOR CHECKING INDEX
748 EXEC sp_helpindex 'Enrollments';
749 -- Index 4 : Composite Index(StudentID,CourseID)
750 CREATE INDEX IX_Enrollments_Student_Course
751 ON Enrollments(StudentID, CourseID);
752 --FOR CHECKING INDEX
753 EXEC sp_helpindex 'Enrollments';
754
755 /*
756 Q1.Why Each Index Improves Performance?
```

The bottom pane shows the 'Results' tab with a table containing index information:

	index_name	index_description	index_keys
1	IX_Enrollments_EnrollmentDate	nonclustered located on PRIMARY	EnrollmentDate
2	IX_Enrollments_Student_Course	nonclustered located on PRIMARY	StudentID, CourseID
3	PK__Enrollme__7F6877FB15A82088	clustered, unique, primary key located on PRIMARY	EnrollmentID

Q1.Why Each Index Improves Performance?

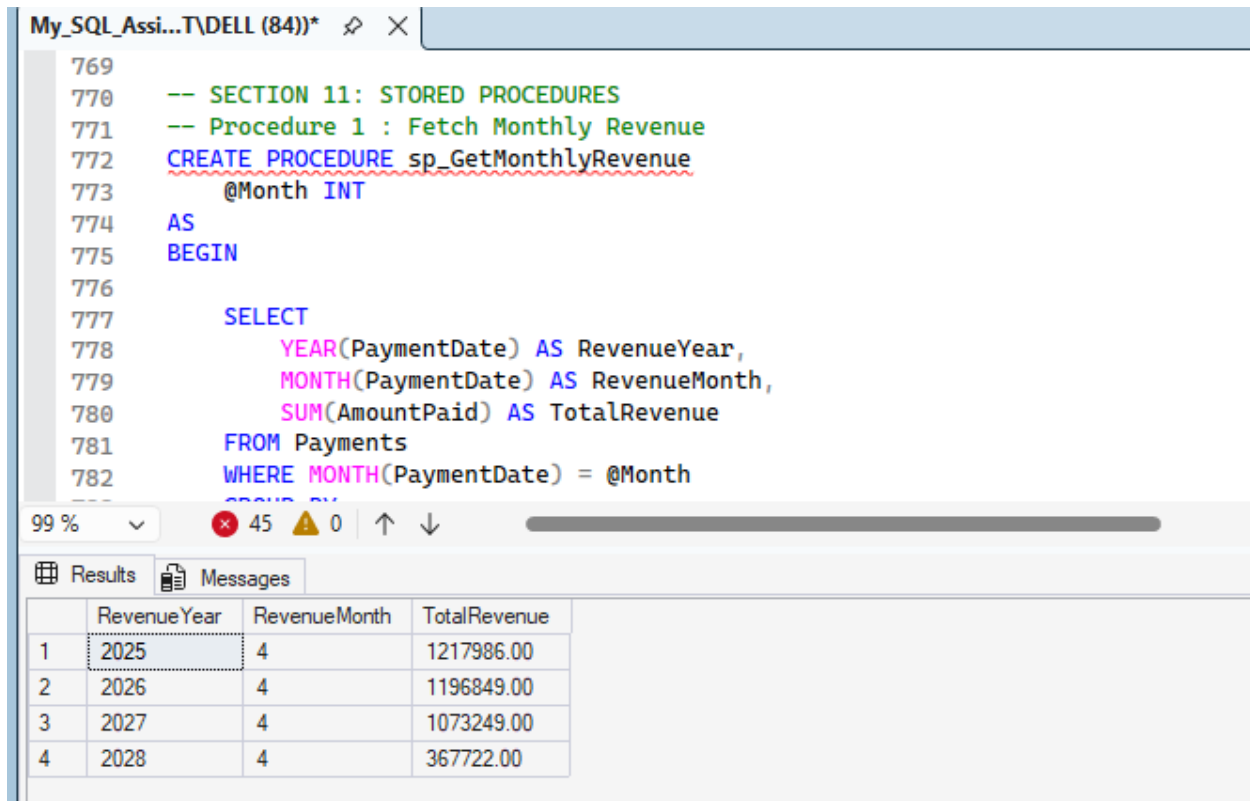
ANS- Indexes improve query performance by allowing SQL Server to locate data quickly without scanning the entire table. They reduce data retrieval time for searching, filtering, sorting, and joining operations. In this project, indexes on Students.Email, Courses.CourseName, Enrollments.EnrollmentDate, and the composite index on (StudentID, CourseID) help optimize common search, reporting, and join queries, making the database more efficient.

Q2.Where Indexing is Unnecessary?

ANS- Indexing is unnecessary on columns that have very few distinct values (low selectivity), such as PaymentStatus (Paid, Unpaid, Partially Paid). These indexes usually provide little performance benefit. Similarly, creating indexes on very small tables or adding too many indexes can reduce performance because SQL Server must update every index during INSERT, UPDATE, and DELETE operations.

PART 11 – STORED PROCEDURES

-- Procedure 1 : Fetch Monthly Revenue



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the execution of a stored procedure named `sp_GetMonthlyRevenue`. The code is as follows:

```
769
770  -- SECTION 11: STORED PROCEDURES
771  -- Procedure 1 : Fetch Monthly Revenue
772  CREATE PROCEDURE sp_GetMonthlyRevenue
773      @Month INT
774  AS
775  BEGIN
776
777      SELECT
778          YEAR(PaymentDate) AS RevenueYear,
779          MONTH(PaymentDate) AS RevenueMonth,
780          SUM(AmountPaid) AS TotalRevenue
781      FROM Payments
782      WHERE MONTH(PaymentDate) = @Month
```

The bottom pane shows the results of the procedure execution. The status bar indicates 99% completion, 45 rows affected, and 0 errors. The Results tab is active, displaying a table with the following data:

	RevenueYear	RevenueMonth	TotalRevenue
1	2025	4	1217986.00
2	2026	4	1196849.00
3	2027	4	1073249.00
4	2028	4	367722.00

-- Procedure 2 : Add New Student with Validation

```
My_SQL_Assi...T\DELL (84))* X
838 );
839
840 PRINT 'Student added successfully.';
841
842 END
843
844 END;
845 --FOR CHECK RUN THIS COMMAND
846 EXEC sp_AddStudent
847
848 @FullName = 'Rahul Sharma',
849 @Email = 'rahul@gmail.com',
850 @Phone = '9876543210',
851 @City = 'Delhi',
852 @State = 'Delhi',
853 @RegistrationDate = '2025-07-15',
854 @IsActive = 1;
855 -- Procedure 3 :Getting Trainer Performance above a threshold(OUT)
856 CREATE PROCEDURE sp_TrainerPerformance
```

99 % 40 0

Messages

(0 rows affected)

(1 row affected)

Unable to add student. Please check the data and try again.

Completion time: 2026-07-24T19:34:37.9772695+05:30

-- Procedure 3 :Getting Trainer Performance above a threshold(OUT)

My_SQL_Assi...T\DELL (84))*

```

885         INNER JOIN TrainerCourseMapping tm
886         ON t.TrainerID = tm.TrainerID
887         GROUP BY
888         t.TrainerID
889         HAVING COUNT(tm.CourseID) >= @MinCourses
890     ) AS T;
891
892 END;
893 --FOR CHECK RUN THIS COMMAND
894 DECLARE @Count INT;
895
896 EXEC sp_TrainerPerformance
897     @MinCourses = 2,
898     @TrainerCount = @Count OUTPUT;
899
900 SELECT @Count AS TotalQualifiedTrainers;
901
902 -- SECTION 12:USER DEFINED FUNCTIONS
903 --Create Scalar Function → fn_NetFeeAfterDiscount

```

99 % 45 0

Results Messages

	TrainerID	TrainerName	TotalAssignedCourses
1	4001	Trainer_1	6
2	4002	Trainer_2	5
3	4006	Trainer_6	5
4	4008	Trainer_8	6
5	4009	Trainer_9	4

PART 12 – USER DEFINED FUNCTIONS

--Create Scalar Function → fn_NetFeeAfterDiscount

My_SQL_Assi...T\DELL (84))* ✨ X

```
903 --Create Scalar Function → fn_NetFeeAfterDiscount
904 CREATE FUNCTION fn_NetFeeAfterDiscount
905 (
906     @CourseFee DECIMAL(10,2),
907     @Discount DECIMAL(5,2)
908 )
909 RETURNS DECIMAL(10,2)
910 AS
911 BEGIN
912
913     RETURN @CourseFee - (@CourseFee * @Discount / 100);
914
915 END;
916 --EXECUTE THIS COMMAND FOR SHOW OUTPUT
917 SELECT dbo.fn_NetFeeAfterDiscount(50000,10) AS NetFee;
918
919 --Create Table-Valued Function → Return Enrollments by Course
920 CREATE FUNCTION fn_EnrollmentsByCourse
921 (
```

99 % 45 0 ↑ ↓

Results Messages

	NetFee
1	45000.00

--Create Table-Valued Function → Return Enrollments by Course

My_SQL_Assi...T\DELL (84))*

```
918
919 --Create Table-Valued Function → Return Enrollments by Course
920 CREATE FUNCTION fn_EnrollmentsByCourse
921 (
922     @CourseID INT
923 )
924 RETURNS TABLE
925 AS
926 RETURN
927 (
928     SELECT
929         e.EnrollmentID,
930         s.StudentID,
931         s.FullName,
932         c.CourseName,
```

99 %

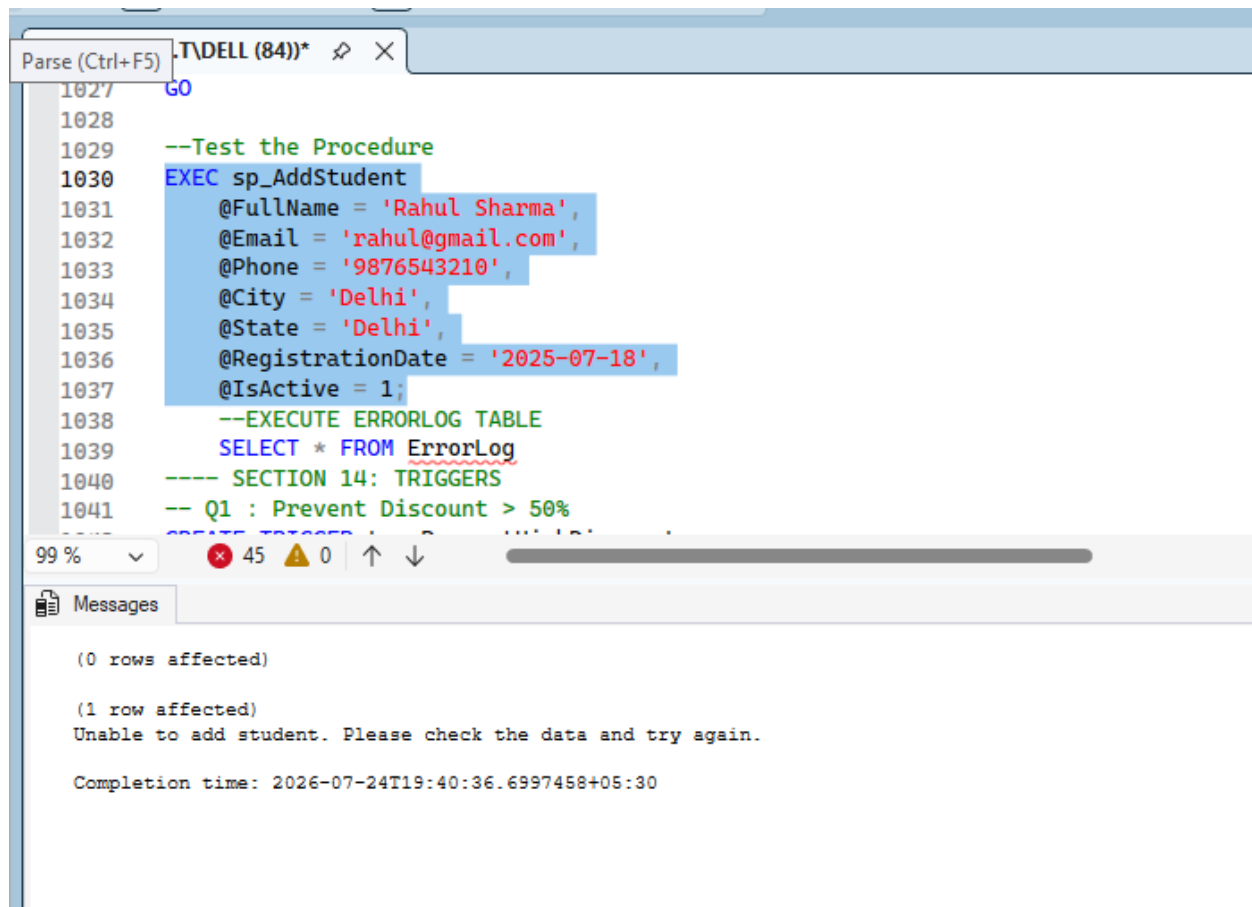
45 0

Results

Messages

	EnrollmentID	StudentID	FullName	CourseName	EnrollmentDate	Discount	PaymentStatus
1	2009	1900	Student_900	Power BI	2025-06-16	25.00	Paid
2	2011	1314	Student_314	Power BI	2025-11-09	20.00	Paid
3	2016	1127	Student_127	Power BI	2025-05-21	25.00	Paid
4	2023	1524	Student_524	Power BI	2025-12-03	20.00	Paid
5	2051	1035	Student_35	Power BI	2025-12-05	5.00	Unpaid
6	2093	1530	Student_530	Power BI	2025-02-02	5.00	Paid
7	2095	1021	Student_21	Power BI	2025-02-19	20.00	Paid
8	2096	1737	Student_737	Power BI	2025-01-19	30.00	Unpaid
9	2102	1478	Student_478	Power BI	2025-05-21	25.00	Partially Paid
10	2115	1381	Student_381	Power BI	2025-11-15	0.00	Unpaid
11	2119	1231	Student_231	Power BI	2025-08-06	30.00	Paid

-- --Create Table-Valued Function → Return Enrollments by Course



The screenshot displays the SQL Server Enterprise Manager interface. The top window, titled 'T\DELL (84))*', contains a SQL script with the following content:

```
1027 GO
1028
1029 --Test the Procedure
1030 EXEC sp_AddStudent
1031     @FullName = 'Rahul Sharma',
1032     @Email = 'rahul@gmail.com',
1033     @Phone = '9876543210',
1034     @City = 'Delhi',
1035     @State = 'Delhi',
1036     @RegistrationDate = '2025-07-18',
1037     @IsActive = 1;
1038 --EXECUTE ERRORLOG TABLE
1039 SELECT * FROM ErrorLog
1040 ---- SECTION 14: TRIGGERS
1041 -- Q1 : Prevent Discount > 50%
```

Below the script, a status bar indicates '99 %' completion, '45' errors, and '0' warnings. The bottom window, titled 'Messages', displays the following output:

```
(0 rows affected)

(1 row affected)
Unable to add student. Please check the data and try again.

Completion time: 2026-07-24T19:40:36.6997458+05:30
```

PART 13 – EXCEPTION HANDLING

--EXECUTE ERRORLOG TABLE

My_SQL_Assi...T\DELL (84))*

```

1027 GO
1028
1029 --Test the Procedure
1030 EXEC sp_AddStudent
1031     @FullName = 'Rahul Sharma',
1032     @Email = 'rahul@gmail.com',
1033     @Phone = '9876543210',
1034     @City = 'Delhi',
1035     @State = 'Delhi',
1036     @RegistrationDate = '2025-07-18',
1037     @IsActive = 1;
1038 --EXECUTE ERRORLOG TABLE
1039 SELECT * FROM ErrorLog
1040 ---- SECTION 14: TRIGGERS
1041 -- Q1 : Prevent Discount > 50%

```

99 % 45 0 Ln: 1039, Ch: 1 (30 char)

Results Messages

	ErrorLogID	ErrorNumber	ErrorMessage	ErrorProcedure	ErrorLine	ErrorDate
1	1	515	Cannot insert the value NULL into column 'Stude...	sp_AddStudent	16	2026-07-24 18:15:50.520
2	2	515	Cannot insert the value NULL into column 'Stude...	sp_AddStudent	16	2026-07-24 19:34:37.967
3	3	515	Cannot insert the value NULL into column 'Stude...	sp_AddStudent	16	2026-07-24 19:40:36.690

PART 14 - TRIGGERS

--AFTER INSERT → Auto insert AmountPaid = 0 when
PaymentStatus = 'Unpaid'

My_SQL_Assi...T\DELL (87)

```

1142     GETDATE(),
1143     10,
1144     'Unpaid'
1145 );
1146 --FOR VERIFYING RUN THIS COMMAND
1147 SELECT *
1148 FROM Payments
1149 ORDER BY PaymentID DESC;
1150
1151 --AFTER UPDATE → Log Old + New Values INTO AUDITLOG TABLE
1152
1153 --First create the audit table.
1154 CREATE TABLE AuditLog
1155 (
1156     AuditID INT IDENTITY(1,1) PRIMARY KEY,

```

99 % 52 0

Results Messages

	PaymentID	EnrollmentID	AmountPaid	PaymentDate
1	4200	3200	49377.00	2028-04-14
2	4199	3199	44194.00	2028-04-13
3	4198	3198	20595.00	2028-04-12
4	4197	3197	28074.00	2028-04-11
5	4196	3196	37381.00	2028-04-10
6	4195	3195	17358.00	2028-04-09
7	4194	3194	8593.00	2028-04-08
8	4193	3193	46936.00	2028-04-07
9	4192	3192	10150.00	2028-04-06
10	4191	3191	22606.00	2028-04-05
11	4190	3190	20982.00	2028-04-04

--AFTER UPDATE → Log Old + New Values INTO AUDITLOG TABLE

```
My_SQL_Assi...T\DELL (87)) X
1181     FROM deleted d
1182     INNER JOIN inserted i
1183         ON d.PaymentID = i.PaymentID;
1184
1185 END;
1186 GO
1187 --FOR TESTING TRIGGER
1188 UPDATE Payments
1189 SET AmountPaid = AmountPaid + 500
1190 WHERE PaymentID = 1;
1191 --FOR CHECKING AUDITLOG TABLE DATA ARE INSERT OR NOT
1192 SELECT *
1193 FROM AuditLog;
1194
1195 -- SECTION 15: FINAL ANALYTICAL INSIGHTS
```

99 % 52 0

Results Messages

AuditID	PaymentID	OldAmount	NewAmount	UpdatedOn
---------	-----------	-----------	-----------	-----------

PART 15 – FINAL ANALYTICAL TASKS

--CAC (Cost Per Acquisition) & ROI per Marketing Channel

My_SQL_Assi...T\DELL (87)

1192 SELECT *
1193 FROM AuditLog;
1194
1195 -- SECTION 15: FINAL ANALYTICAL INSIGHTS
1196 --CAC (Cost Per Acquisition) & ROI per Marketing Channel
1197 SELECT
1198
1199 Channel,
1200
1201 SUM(Spend) AS TotalSpend,
1202
1203 SUM(LeadsGenerated) AS TotalLeads,
1204
1205 ROUND(
1206 SUM(Spend) * 1.0 / SUM(LeadsGenerated),

99 % 52 0 ↑ ↓

Results Messages

	Channel	TotalSpend	TotalLeads	CAC
1	Google Ads	1696089.00	8719	194.530000
2	Meta Ads	1926665.00	9871	195.180000
3	Influencers	2300870.00	11513	199.850000
4	Email	1728848.00	8111	213.150000
5	YouTube	2063094.00	8912	231.500000
6	LinkedIn	2297856.00	9743	235.850000

--12 Month Enrollment Heatmap (Month × City)

My_SQL_Assi...T\DELL (87)

```

1213
1214 ORDER BY CAC;
1215
1216 --12 Month Enrollment Heatmap (Month x City)
1217 SELECT
1218     DATENAME(MONTH,e.EnrollmentDate) AS Month,
1219     s.City,
1220     COUNT(*) AS TotalEnrollments
1221 FROM Students s
1222 INNER JOIN Enrollments e

```

99 % 52 0

Ln: 1

Results Messages

	Month	City	TotalEnrollments
1	January	Ahmedabad	8
2	January	Bangalore	8
3	January	Chennai	13
4	January	Delhi	9
5	January	Goa	11
6	January	Hyderabad	6
7	January	Jaipur	6
8	January	Kolkata	12
9	January	Mumbai	6
10	January	Pune	5
11	February	Ahmedabad	7
12	February	Bangalore	6
13	February	Chennai	6
14	February	Delhi	10
15	February	Goa	10
16	February	Hyderabad	15

Query executed successfully. DESKTOP-V9S50FT\SQLEXPRESS ... DESKTOP-V9S50FT\DELL (87)

--Top Course & Worst Course by Revenue

1230	S.City,
1231	
1232	--Top Course & Worst Course by Revenue
1233	SELECT
1234	c.CourseName,
1235	SUM(p.AmountPaid) AS TotalRevenue
1236	FROM Courses c
1237	INNER JOIN Enrollments e
1238	ON c.CourseID = e.CourseID
1239	INNER JOIN Payments p
1240	ON e.EnrollmentID = p.EnrollmentID
1241	GROUP BY
1242	c.CourseName
1243	ORDER BY
1244	TotalRevenue DESC;

99 %	52	0	↑	↓
------	----	---	---	---

Results	Messages
---------	----------

	CourseName	TotalRevenue
1	Data Analytics	4747754.00
2	Power BI	4469093.00
3	Data Science	4458602.00
4	Generative AI	4334813.00
5	SQL Mastery	4261140.00
6	Cloud Found...	4244354.00
7	Python Full ...	4143598.00
8	Machine Le...	3880273.00
9	Business An...	3741912.00
10	Excel Power...	3476370.00

Top row = Best course

Last row = Worst course

--Trainer Impact on Course Satisfaction

My_SQL_Assi...T\DELL (87)) ✕

```

1249
1250 --Trainer Impact on Course Satisfaction
1251 SELECT
1252     t.TrainerName,
1253     AVG(t.SatisfactionScore) AS AverageSatisfaction,
1254     COUNT(tcm.CourseID) AS AssignedCourses
1255 FROM Trainers t
1256 INNER JOIN TrainerCourseMapping tcm
1257 ON t.TrainerID = tcm.TrainerID
1258 GROUP BY
1259     t.TrainerName
1260 ORDER BY
1261     AverageSatisfaction DESC;
1262
1263 --Month over month bussiness growth

```

99 % 52 0 ↑ ↓

Results Messages

	TrainerName	AverageSatisfaction	AssignedCourses
1	Trainer_9	4.480000	4
2	Trainer_4	4.410000	1
3	Trainer_3	4.400000	1
4	Trainer_6	4.340000	5
5	Trainer_1	4.190000	6
6	Trainer_10	3.990000	1
7	Trainer_5	3.870000	1
8	Trainer_8	3.740000	6
9	Trainer_2	3.500000	5

--Month over month bussiness growth

1278	OVER
1279	(
1280	ORDER BY YEAR(PaymentDate), MONTH(PaymentDate)
1281	) AS RevenueGrowth
1282	
1283	FROM Payments
1284	
1285	GROUP BY
1286	YEAR(PaymentDate),
1287	MONTH(PaymentDate)
1288	
1289	ORDER BY
1290	Year,
1291	Month;
1292	

99 %	52	0	↑	↓
------	----	---	---	---

Results	Messages
---------	----------

	Year	Month	Revenue	PreviousMonthRevenue	RevenueGrowth
1	2025	1	1080815.00	NULL	NULL
2	2025	2	893113.00	1080815.00	-187702.00
3	2025	3	1211624.00	893113.00	318511.00
4	2025	4	1217986.00	1211624.00	6362.00
5	2025	5	999518.00	1217986.00	-218468.00
6	2025	6	974654.00	999518.00	-24864.00
7	2025	7	1130095.00	974654.00	155441.00
8	2025	8	1236910.00	1130095.00	106815.00
9	2025	9	1004515.00	1236910.00	-232395.00
10	2025	10	1205190.00	1004515.00	200675.00
11	2025	11	1003845.00	1205190.00	-201345.00

PART 16 – FINAL INSIGHTS REPORT

1. Revenue Trends

Revenue showed month-to-month variation throughout the year. Monthly revenue analysis helps identify periods of business growth and slower performance. Monitoring these trends enables better financial planning and forecasting.

2. Marketing Effectiveness

The CAC (Cost per Acquisition) analysis showed that some marketing channels generated leads more efficiently than others. Channels with lower CAC provide better value for marketing spend and should receive greater focus in future campaigns. Because the database does not link marketing channels directly to payments, true ROI by channel cannot be calculated from the available data.

3. High Performing Courses.

Revenue analysis identified the highest-performing courses based on total payment collections. These courses attract more enrollments and generate the highest revenue. Courses with lower revenue may require updated content, improved marketing, or pricing adjustments to increase enrollment.

4. City-wise Demand

The Month $\times$ City enrollment analysis showed that student demand varies across different cities. Cities with consistently higher enrollment should be prioritized for future marketing campaigns, while cities with lower enrollment may require additional promotional activities to improve student acquisition.

5. Trainer Impact

Trainer performance analysis showed that trainers with higher satisfaction scores generally contribute to better student satisfaction. Trainers handling multiple courses with consistently high ratings demonstrate strong teaching effectiveness and should be considered for additional course assignments.

6. Overall Business Recommendations

- Continue investing in marketing channels with the lowest Cost per Acquisition (CAC).
- Promote the highest-revenue courses through targeted marketing campaigns.
- Improve or redesign low-performing courses to increase enrollments.
- Focus marketing efforts on cities with higher student demand while developing strategies to improve enrollments in lower-performing regions.
- Regularly monitor trainer satisfaction and provide training programs to maintain high teaching quality.
- Track monthly revenue and enrollment trends to support data-driven business decisions